第 21 章：取消、超时与截止时间

先记住一句话

`asyncio` 的取消不是“立刻杀死任务”，而是向 Task 发出请求，让 `CancelledError` 在后续的协作调度点进入协程；超时通常也是先触发这套取消协议，再由超时边界把结果解释为 `TimeoutError`。

Agent 收到“停止”后，真正需要的不是返回一个“已取消”字符串，而是让整条等待链停止继续产生结果，同时仍然释放连接、会话和锁。

这两个概念最容易被一句“cancel 就停了”掩盖。实际上至少要分开四件事：

```
调用方发出 cancel 请求
        ↓ 下一次事件循环推进目标 Task
CancelledError 注入等待链
        ↓
协程执行 finally（可能还包含异步清理）
        ↓
Task 最终结束；等待者观察到取消、超时、结果或其他异常
```

完成本章后，你应该能够：

- 解释 `Task.cancel()` 为什么只是请求，不是同步终止命令；
- 区分取消请求、异常注入、清理中和 Task 最终取消四个阶段；
- 沿“入口 → 工具 → I/O”逐层说出 `CancelledError` 的传播方向；
- 正确选择 `asyncio.timeout()`、`timeout_at()` 和 `wait_for()`；
- 用同一个单调时钟截止时间约束单次尝试、单工具和整个 Agent 请求；
- 解释事件循环阻塞、慢清理和不可撤回的外部副作用为何会突破直觉中的“准点停止”；
- 识别吞掉取消和遗留子 Task 的生命周期缺陷。

`asyncio.shield()` 放在选学部分。它不是常规取消修复方式，只适用于调用方明确拥有、愿意在自己取消后继续管理的工作。

Java 与 Python 心智模型

Java → Python 心智模型

可以先把 Python Task 的取消类比为 Java 中“ `Future` 生命周期 + 线程中断意图 + 异常传播”的组合，但不能完全等同。Task 通常不是一条独占线程， `Task.cancel()` 也不会调用 `Thread.interrupt()` 。

Java 中熟悉的概念	Python 中可以先想到	最重要的不等价之处
`Future.cancel(true)`	`task.cancel()`	Java 可尝试中断执行线程；Python 不做线程中断，而是安排 `CancelledError`
`InterruptedException` / `CancellationException`	`asyncio.CancelledError`	Python 取消在 Task 的协作调度点传播，不是线程中断标志
`future.get()`	`await task`	两者都观察最终结果；Python 的 `await` 还会形成取消传播边
`Future.get(timeout, unit)`	`asyncio.wait({task}, timeout=...)`	都不自动取消工作；Java 抛异常，Python 返回 done/pending 两个集合
带超时并取消目标	`asyncio.wait_for(target, seconds)`	`wait_for()` 到期会取消目标，并等待目标完成取消
给一段调用链设置预算	`async with asyncio.timeout(seconds)`	它取消执行代码块的当前 Task，不会自动创建 worker Task
`System.nanoTime()` 计算剩余预算	`loop.time()` + `timeout_at()`	都适合经过时间；都不是日历时间或 Unix 时间戳
`try/finally` 、 `try-with-resources`	`try/finally` 、异步上下文管理器	Python 的清理还能包含 `await` ，取消后 Task 仍可能是 pending
调用方保存并回收 `Future`	Task 所有者保存并 `await` Task	只保存引用不等于已经等待清理完成

Java 的 `Future.cancel(...)` 也提醒调用方不要只靠返回值判断状态，而应查看 `isCancelled()` 。Python 的差异更深一层： `task.cancel()` 只是安排异常进入协程，只有 coroutine 最终以 `CancelledError` 结束时， `task.cancelled()` 才会为 `True` 。

异常层级也不能照搬。Java 的 `InterruptedException` 是受检 `Exception` ，

异常层级也不能照搬。Java 的 `InterruptedException` 是受检 `Exception`，`CancellationException` 是运行时异常；Python 的 `asyncio.CancelledError` 则直接继承 `BaseException`，故意绕开普通的 `except Exception` 业务错误边界。

21.1 前置、环境与运行方式

本章承接第 18 章的 `try/finally` 与资源所有权，以及第 20 章的 Task 生命周期与结果收集。这里只复习理解本章所需的最小 Task 前置：

- coroutine 对象保存一次异步调用的执行状态；
- `asyncio.create_task(coroutine)` 把它包装成由事件循环调度的 `Task`；
- `Task` 保存最终结果、异常或取消状态，可以被 `await`；
- 创建 Task 的代码通常是它的所有者，应保存引用并最终收集它的结果或异常；
- 直接 `await coroutine` 沿当前 Task 的调用链执行；`create_task()` 才建立另一个 Task。

根 `pyproject.toml` 声明 Python `>=3.14`，Pyright 和 Ruff 目标也都是 Python 3.14。本次优化时核对的实际环境为：

项目	实际值	用途
CPython	3.14.7	运行完整示例与运行时断言
Ruff	0.16.5	格式和常规代码检查
Pyright	1.1.411	基础静态类型检查

本章只使用 `asyncio`、`time` 等标准库，没有新增运行依赖或开发依赖。所有关键 API 都链接到 Python 3.14 官方文档。`asyncio.timeout()` / `timeout_at()` 从 Python 3.11 起提供；本项目只需按 3.14 语义学习，不需要兼容更旧解释器。

从仓库根目录独立运行：

```
.venv/bin/python -B java_to_python/chapters/21_asyncio_cancellation_timeouts/example.py
```

`-B` 只阻止生成 `.pyc`，不改变示例行为。现有 `java_to_python/run_all.py` 只支持前 10 章，本章没有接入它。

项目已配置且当前虚拟环境可用的检查命令是：

README × | readme × | README × | README × | README × | README × | README × | README × | README
7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f5f43d449c8478027cdad447ae1.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

```
.venv/bin/ruff check java_to_python/chapters/21_asyncio_cancellation_timeouts
.venv/bin/ruff format --check java_to_python/chapters/21_asyncio_cancellation_timeouts
.venv/bin/pyright java_to_python/chapters/21_asyncio_cancellation_timeouts/example.py
```

21.2 `cancel()` 发出什么请求

先看一个可以独立运行的最小取消骨架：

```python
import asyncio


async def worker() -> None:
    try:
        await asyncio.Event().wait()
    finally:
        print("释放资源")


async def main() -> None:
    task = asyncio.create_task(worker())
    await asyncio.sleep(0)  # 让 worker 先运行到等待点
    accepted = task.cancel("用户停止")
    print(accepted, task.done(), task.cancelled())  # True False False
    try:
        await task
    except asyncio.CancelledError as error:
        print(error.args)  # ('用户停止',)


asyncio.run(main())
```

Python 3.14 的 `Task.cancel()`
定义是：安排一个 `CancelledError`，在事件循环下一次推进这个 Task 时抛入其包装的 coroutine。
因此它有几个重要限制：

1. `cancel()` 不会在当前 Python 表达式中抢占另一个 Task；
2. 目标 Task 要先获得事件循环执行机会，取消才会被处理；
3. 若目标正在等待 Future，取消请求通常先传给这个被等待对象；
4. coroutine 可以在 `except / finally` 中执行清理，因而不会立刻结束；

1. `cancel()` 不会在当前 Python 表达式中抢占另一个 Task；
2. 目标 Task 要先获得事件循环执行机会，取消才会被处理；
3. 若目标正在等待 Future，取消请求通常先传给这个被等待对象；
4. coroutine 可以在 `except / finally` 中执行清理，因而不会立刻结束；
5. coroutine 甚至可以错误地吞掉 `CancelledError` 、返回值或抛出另一种异常，所以取消不保证最终状态一定是 cancelled。

`cancel()` 的布尔返回值只说明请求是否被接受。对一个已完成 Task 再调用通常返回 `False` ；返回 `True` 仍不能替代 `await task` 。 `task.cancelling()` 可观察待处理的取消请求数量，也不表示清理已经结束。

这里不能照搬 Java `Future.cancel(true)` 的实现模型。Java 可以尝试中断执行任务的线程；Python Task 还必须获得事件循环执行机会，并让 `CancelledError` 通过 coroutine。协程可以运行清理，甚至错误地吞掉取消，因此一定要观察 Task 的最终状态。

`CancelledError` 通常在哪里出现

如果目标 Task 正暂停在一个尚未完成的 `await` ：

```
await response_ready.wait()
```

事件循环再次推进它时，这个 `await` 会以 `CancelledError` 离开，异常再沿普通 Python 调用栈向外传播。它不是操作系统在任意机器指令处插入的硬中断。

如果 coroutine 正在连续执行 Python 代码、调用 `time.sleep()` 、运行同步 HTTP 客户端或做长 CPU 循环，事件循环得不到执行权，取消注入就会推迟。 `await` 也不是无条件切换点：等待一个已经完成的对象可能立即返回；决定响应性的根本仍是代码是否把执行机会交回事件循环。

这和 Java 线程收到 interrupt 后仍需要代码或阻塞 API 配合响应很像，但 Python Task 没有被强行停止的独占线程，也没有任意机器指令级抢占。

官方的总体清理建议见 `Task cancellation` 。

21.3 四个阶段不能混成一句“任务取消了”

要准确观察这四个阶段，可以先用一个 `Event` 确认模拟 I/O 已经开始，再发出用户取消；异步清理再由另一个 `Event` 暂停。这样观察的是明确状态，而不是依靠 `sleep` 猜测调度时序：

README × | readme × | README × | README × | README × | README × | README × | README × | README
f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f5f43d449c8478027cdad447ae1.html
关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

21.3 四个阶段不能混成一句“任务取消了”

要准确观察这四个阶段，可以先用一个 `Event` 确认模拟 I/O 已经开始，再发出用户取消；异步清理再由另一个 `Event` 暂停。这样观察的是明确状态，而不是依靠 `sleep` 猜测调度时序：

阶段	done()	cancelled()	资源状态	能得出的结论
`cancel()` 刚返回	`False`	`False`	`OPEN`	请求已登记，异常尚未注入
I/O 收到 `CancelledError`	`False`	`False`	`OPEN`	取消正在沿调用链传播
`finally` 异步清理中	`False`	`False`	`OPEN`	不能开始依赖“资源已关”的下一步
`await task` 抛取消后	`True`	`True`	`RELEASED`	任务以取消终止，清理已完成

这里的最终 `cancelled() is True` 有一个前提：包装的 coroutine 最终以 `CancelledError` 结束。如果它捕获后返回普通值，Task 会是“成功完成”，`cancelled()` 为 `False`，尽管调用方确实调用过 `cancel()`。

为什么必须 `await` 被取消的 Task

```
task.cancel()
await task
```

第二行不是重复操作。两行职责不同：

- `cancel()` 发请求；
- `await task` 等到最终结果，观察取消或其他异常，并与 `finally` 清理完成建立顺序关系。

把 Task 存在列表、字典或对象字段里很重要，因为所有者需要强引用和后续管理入口；但“引用还在”只说明对象可找到，不说明它已完成。只保存引用、不在关闭边界真正等待，后续代码仍可能在资源清理期间继续运行，程序退出时也可能留下 pending Task。

21.4 三层调用链：记录以后要继续抛

本章的核心链路是：

```
request_entry()                 入口层
```

README × | readme × | README × | README × | README × | README × | README × | README × | README
vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f5f43d449c8478027cdad447ae1.html
2.50
相关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单计数

21.4 三层调用链：记录以后要继续抛

本章的核心链路是：

```
request_entry()                入口层
  └ await call_tool()          工具层
      └ await simulated_io()   模拟 I/O 层
          └ finally 释放资源
```

三个层次都可以为了日志、指标或局部清理观察取消，但不应把它改写成普通业务结果：

```
async def call_tool() -> str:
    try:
        return await simulated_io()
    except asyncio.CancelledError:
        logger.info("工具调用被取消")
        raise  # 让取消继续到拥有这条调用链的边界
```

多数函数甚至不必捕获取消；把资源清理放在 `finally`，让异常自然传播更不容易出错：

```
resource = open_resource()
try:
    return await use_resource(resource)
finally:
    resource.release()
```

如果清理本身包含 `await`，Task 在清理完成前仍是 pending。清理应尽可能有限、幂等，并能处理部分完成。另一次新的取消仍可能打断异步清理；复杂系统应由清楚的生命周期所有者设计关闭策略，而不是在每一层随意屏蔽取消。

21.5 `CancelledError` 为什么不属于普通业务异常

Python 3.8 起，`asyncio.CancelledError` 直接继承 `BaseException`，而不是 `Exception`。Python 3.14 的正式层级见 `asyncio.CancelledError`。

```
BaseException
```

README × | readme × | README × | README × | README × | README × | README × | README × | README
vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f5f43d449c8478027cdad447ae1.html
2.50
目关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

21.5 `CancelledError` 为什么不属于普通业务异常

Python 3.8 起， `asyncio.CancelledError` 直接继承 `BaseException` ，而不是 `Exception` 。Python 3.14 的正式层级见 `asyncio.CancelledError` 。

```
BaseException
├─ SystemExit
├─ KeyboardInterrupt
├─ GeneratorExit
├─ asyncio.CancelledError
└─ Exception
   ├─ ValueError
   ├─ RuntimeError
   └─ 大多数业务与库异常
```

因此各种捕获写法的范围不同：

写法	会捕获 `CancelledError` 吗	本章建议
`except Exception:`	不会	适合普通错误边界， 但仍要避免不加分析地吞业务错误
`except asyncio.CancelledError:`	会	只在记录或取消专用清理时使用，通常裸 `raise`
`except BaseException:`	会	只适合“清理后原样重抛”等很窄的基础设施边界
裸 `except:`	会	范围同样过宽，还会捕获退出信号；通常不要写

这里尤其不能套用 Java 的 `catch (Exception e)` 直觉。Java 的 `InterruptedException` 会被它捕获，Python 的 `except Exception` 却不会捕获 `CancelledError` ；只有更宽的 `except BaseException` 或裸 `except` 才会把取消一起拦下。

`except Exception` 不会误吞取消是有意的安全边界，但这不表示所有宽泛捕获都安全。下面两个反例都会破坏结构化取消：

```
# 教学反例 1：把取消伪装成业务失败。
try:
```

```
# 教学反例 1：把取消伪装成业务失败。
try:
    return await call_remote_tool()
except BaseException as error:
    return {"ok": False, "error": str(error)}


# 教学反例 2：记录后继续下一轮，用户已停止但工具仍会产生副作用。
try:
    await call_remote_tool()
except asyncio.CancelledError:
    logger.info("stopped")
    # 漏了 raise

await submit_side_effect()
```

一个受控反例可以在取消被吞后，由外部打开本地 Event 闸门让它有限结束。最终 Task 返回“伪成功”、`cancelled()` 为 `False`，甚至继续提交副作用。正确修复是清理后继续抛出取消，而不是在每个函数中 catch 后 continue。

21.6 直接 `await` 另一个 Task 时怎样传播

假设父 Task 正在直接等待子 Task：

```
child = asyncio.create_task(call_tool())
result = await child
```

当父 Task 被取消时，它正在等待的对象也会收到取消。子 Task 是 Future-like 对象，因此取消沿等待边向内传播：

```
cancel(parent)
   ↓
parent 正在 await child
   ↓
cancel(child)
   ↓
child 的 coroutine 收到 CancelledError 并执行 finally
   ↓
child 以取消结束
```

```
cancel(parent)
   ↓
parent 正在 await child
   ↓
cancel(child)
   ↓
child 的 coroutine 收到 CancelledError 并执行 finally
   ↓
child 以取消结束
   ↓
parent 的 await 抛出 CancelledError，parent 再向外传播
```

即使子 Task 的引用一直保存在 `child_task` 变量里，清理闸门关闭时 `child_task.done()` 仍然为 `False`。只有打开闸门并真正等待父 Task 后，父子 Task 才都完成取消，资源也已经释放。

这段结论以子 coroutine **正常响应取消并重新抛出**为前提。若子任务吞掉取消，它可能返回值或继续等待，父任务完成时间和最终异常也会受影响。若不希望父取消传播到子任务，那已进入 `shield()` 的特殊所有权场景，而不是默认行为。

21.7 `asyncio.timeout()`：给一段当前调用链划范围

正确捕获结构是：

```
try:
    async with asyncio.timeout(2.0):
        await call_tool()
except TimeoutError:
    print("这段操作超过 2 秒")
```

`asyncio.timeout()` 是异步上下文管理器。进入时它记住当前 Task；到期时取消的也是 **当前 Task**，不是自动创建的独立 worker。其运行过程是：

1. 当前 Task 进入 timeout 范围；
2. 截止时间到达，范围向当前 Task 发出取消；
3. 最深处尚未完成的 `await` 收到 `CancelledError`；
4. `CancelledError` 沿当前调用链向外传播，途中先完成 `finally`；
5. timeout 管理器在退出范围时识别自己引发的取消；

1. 当前 Task 进入 timeout 范围；
2. 截止时间到达，范围向当前 Task 发出取消；
3. 最深处尚未完成的 `await` 收到 `CancelledError`；
4. `CancelledError` 沿当前调用链向外传播，途中先完成 `finally`；
5. timeout 管理器在退出范围时识别自己引发的取消；
6. 范围外看到内置 `TimeoutError`。

它没有严格对应的单个 Java API。可以把它理解为"作用域级 deadline + 对当前执行单元发出取消请求 + 在边界转换异常"的组合，而不是一次带超时的 `Future.get(...)`。

所以由这个管理器产生的 `TimeoutError` 只能在 `async with` 外面捕获：

```
# 教学反例：这里捕不到当前 timeout 管理器最终转换出的 TimeoutError。
async with asyncio.timeout(2.0):
    try:
        await call_tool()
    except TimeoutError:
        ...
```

范围里面在取消传播阶段看到的是 `CancelledError`。若为了日志捕获它，仍应 `raise`，让管理器有机会在范围边界正确转换。不要自己把所有 `CancelledError` 都写成 `TimeoutError`，因为取消也可能来自用户停止、服务关闭或父任务失败。

本章还验证了相反路径：进入一个尚有 60 秒的 timeout 范围后，由外部调用 `task.cancel()`。
因为该范围并未到期，这次普通外部取消离开范围后仍是 `CancelledError`，没有被误报成超时。
如果外部取消与截止到期恰好竞争，应把语义交给各自清楚的所有权边界，不依赖偶然调度顺序。

`TimeoutError` 的版本名称

Python 3.11 起，asyncio 超时 API 抛内置 `TimeoutError`；`asyncio.TimeoutError` 是它的别名。
3.14 新代码直接写：

```
except TimeoutError:
    ...
```

官方版本说明见
`asyncio.TimeoutError`。

README × readme × README × README × README × README × README × README × README
/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f5f43d449c8478027cdad447ae1.html
2.60
相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

21.8 `wait_for()`：给一个明确目标设置相对超时

```
result = await asyncio.wait_for(one_attempt(), timeout=0.8)
```

`asyncio.wait_for()` 关注的
是一个 awaitable：

- Python 3.14 中，正 timeout 下传入 coroutine 时，不会先把它包装成独立 Task，而是在当前等待路径中直接执行；这是 Python 3.12 起的变化；
- 若要保存任务身份、检查状态或让其他代码管理它，应先显式 `create_task()`，再把 Task 传入；
- 若按时完成，返回它的结果；
- 若等待超时，取消 awaitable，并通常向等待者抛 `TimeoutError`；
- 若 `wait_for()` 自己被取消，取消也传播给目标；
- 它会等待目标实际完成取消，所以总墙钟时间可能超过传入的 `timeout`；
- 若目标错误地吞掉取消并返回值，Python 3.14 的 `wait_for()` 会返回那个值，而不是保证抛 `TimeoutError`。

如果传入一个已有的 `target_task`，并故意让它的 `finally` 等待清理闸门，计时到期后可以观察到：

```
target 已收到取消并进入 finally
target.done()      == False
wait_for.done()    == False
资源               == OPEN
```

打开闸门后，目标释放资源并以取消结束；这时 `wait_for()` 才向调用方报告 `TimeoutError`。因此“0.8 秒超时”通常是“不再愿意等待正常结果的预算”，不是“函数保证在第 0.800 秒已经返回”。

这也是它与 Java `Future.get(800, MILLISECONDS)` 最重要的差异：Java 的限时 `get` 默认只是让当前等待者超时，Future 仍可继续运行；Python 的 `wait_for()` 会主动取消目标。如果只想停止等待并保留 pending Task，应使用 `asyncio.wait()`。它在超时时不抛 `TimeoutError`，而是返回 done/pending 两个集合；调用方仍要继续承担 pending Task 的生命周期管理。

`timeout()` 与 `wait_for()` 怎样选

需求	优先选择	作用对象

`timeout()` 与 `wait_for()` 怎样选

需求	优先选择	作用对象
限制一个明确 awaitable 或一次尝试	`wait_for(aw, seconds)`	目标 awaitable；超时会取消它
限制多条 `await` 、循环和解析组成的范围	`async with timeout(seconds)`	执行该范围的当前 Task
多层代码共享同一个截止时刻	`async with timeout_at(deadline)`	当前 Task，使用绝对单调时钟
只想等一段时间、超时后保留 pending	`asyncio.wait(..., timeout=...)`	返回 done/pending，不自动取消 pending；属于 Task 收集主题

不要仅凭函数名选择。先画出"哪个 Task 执行范围、哪个 awaitable 是目标、超时后谁取消和谁回收"。

21.9 相对预算会偷偷膨胀，绝对截止时间不会

下面的错误设计让每层都重新获得完整 10 秒：

```
# 教学反例：每次调用都从"现在"重新计算 10 秒。
async with asyncio.timeout(10):  # Agent 请求
    async with asyncio.timeout(10):  # 工具
        await asyncio.wait_for(attempt(), 10)  # 单次尝试
```

如果还有重试、退避、多个工具和清理，整个请求很容易远超调用方承诺的预算。正确模型与 Java 使用 `System.nanoTime()` 传递剩余预算相似：先在最外层计算一次绝对截止时间，再向内传递同一个数：

```
loop = asyncio.get_running_loop()
request_deadline = loop.time() + 10.0

try:
    async with asyncio.timeout_at(request_deadline):
        answer = await run_tool(request_deadline=request_deadline)
except TimeoutError:
```

`loop.time()` 返回事件
循环使用的单调时钟；它适合计算经过时间和 deadline，不受系统墙钟校准影响。它的起点没有日历意义，不能拿来与 `time.time()` 、Unix 时间戳或 `datetime.now()` 直接比较。

`asyncio.timeout_at(deadline)`
接收同一时钟体系下的绝对时刻。内层预算可以更严格，但不能更宽：

```
loop = asyncio.get_running_loop()
tool_deadline = min(request_deadline, loop.time() + 2.0)
attempt_deadline = min(tool_deadline, loop.time() + 0.8)
attempt_timeout = max(0.0, attempt_deadline - loop.time())
```

三层职责应保持清楚：

层次	示例限制	失败后通常做什么
单次尝试	最多 0.8 秒，且不越过上层 deadline	目标取消并清理；剩余预算足够时才考虑下一次尝试
单工具预算	搜索工具总共最多 2 秒	停止该工具的所有尝试， 向工具调用边界报告预算耗尽
Agent 请求 deadline	整个请求最多 10 秒	停止继续编排其他工具，向请求边界报告整体超时

每次重试前都要从 **原 deadline** 重新计算剩余时间：

```
remaining = tool_deadline - loop.time()
if remaining <= 0:
    raise TimeoutError("工具总预算已耗尽")

attempt_timeout = min(0.8, remaining)
result = await asyncio.wait_for(one_attempt(), timeout=attempt_timeout)
```

这不是让重试“获得剩余时间之外的新预算”。第一次尝试和清理已经消耗的时间不会被忘掉。

只让更严格的内层创建自己的超时范围

若工具的局部 deadline 与请求 deadline 完全相同，在相同时刻嵌套两个超时范围没有额外保护，还会让“是谁的预算先耗尽”难以命名。本章示例采用：

只让更严格的内层创建自己的超时范围

若工具的局部 deadline 与请求 deadline 完全相同，在相同时刻嵌套两个超时范围没有额外保护，还会让"是谁的预算先耗尽"难以命名。本章示例采用：

- 工具 deadline 更早：工具创建 `timeout_at(tool_deadline)`，并可在范围外转换成 `ToolBudgetExceeded`；
- 工具 deadline 不更早：工具不再复制范围，直接沿用外层请求 deadline；
- 无论哪条路径，外部 `CancelledError` 都不由工具伪装成超时。

嵌套 timeout 管理器本身受官方支持，但业务层仍要决定每个边界代表什么。不要依赖两个完全相同到期时刻的偶然回调顺序来分类错误。

21.10 超时为何不承诺精确到点停止

取消是协作协议，超时又建立在取消上，所以它不是硬中断。三个边界尤其重要。

1. 事件循环被阻塞

```
async with asyncio.timeout(0.01):
    time.sleep(1)  # 错误：事件循环无法执行超时回调
    await something()
```

计时器可能已经到期，但唯一的事件循环线程还卡在 `time.sleep()`。用一个有限的同步阻塞即可观察到"同步阻塞先完成，之后才出现 `TimeoutError`"；可靠的测试应该断言事件顺序，而不是某个精确毫秒数。

2. 清理阶段需要时间

取消进入 coroutine 后，`finally` 会先运行。关闭协议、刷新缓冲、结束事务或等待子 Task 都可能耗时。`wait_for()` 明确会等待目标实际完成取消，`timeout()` 也要等当前调用链离开它的范围后才能在外侧抛 `TimeoutError`。

预算设计应预留清理余量，并在监控中区分：deadline 到期时刻、开始取消时刻、清理结束时刻和调用返回时刻。不要把一个 timeout 数字当成端到端延迟的硬上界。

3. 底层工作可能不可取消或副作用不可撤回

README × readme × README × README × README × README × README × README × README × READ
7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-f5f43d449c8478027cdad447ae1.html
求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

3. 底层工作可能不可取消或副作用不可撤回

取消 Python Task 只改变本地等待与协程控制流，不自动提供以下保证：

- 已在线程中开始的同步函数会立刻停止；
- 已发出的 HTTP 请求一定从服务器队列撤回；
- 数据库已经提交的事务自动回滚；
- MCP tool 已执行的外部动作被逆转；
- 第三方客户端一定以与本地替身相同的方式关闭连接。

原生异步客户端收到取消后可能关闭 socket、发送协议级取消，或只是停止本地等待；具体行为取决于库版本、传输协议和远端实现。需要幂等键、事务、补偿操作或服务端 deadline 时，应把它们当作独立的系统设计，而不是期待 `Task.cancel()` 撤销现实世界。

这和 Java 中断线程不能自动回滚已提交事务是同一个工程边界。两边都需要单独设计幂等与补偿，不能把本地控制流已经停止当作远端副作用已经撤回。

本章按课程边界不运行真实 `asyncio.to_thread()` 或 HTTP 实验；这些放到第 25 章。本地 `FakeIOResource` 只验证 Python 层的传播和清理顺序，没有覆盖线程停止、DNS、TLS、socket、连接池、服务器状态或远程副作用。

21.11 选学：`shield()` 保护谁

默认直接等待会传播取消。有时调用方取消了，但某项已经开始的工作必须完成，例如一个由当前组件负责持久化、且中断比完成更危险的短操作。此时可以考虑：

```
child = asyncio.create_task(critical_work())
try:
    result = await asyncio.shield(child)
except asyncio.CancelledError:
    # 当前调用方仍然被取消；child 没有因这次等待者取消而被取消。
    result = await child  # 当前函数若是所有者，就要决定并收集最终状态
    raise
```

`asyncio.shield()` 保护的是传入的 awaitable，不是外面的调用方：

- 调用方仍会在 `await shield(...)` 处收到 `CancelledError`；
- 这一次调用方取消不会向内取消 child；

- 调用方仍会在 `await shield(...)` 处收到 `CancelledError`；
- 这一次调用方取消不会向内取消 child；
- child 若被其他代码取消， `shield()` 仍会反映那个取消；
- 必须保存 child 的强引用；事件循环只保留弱引用；
- 创建 child 的所有者仍要决定让它完成还是主动取消，并最终 `await` 以收集结果或异常。

如果 owner 取消后仍收集 child，再重新抛出自己的取消，就会同时得到"child 正常完成"和"owner 最终保持 cancelled"两个结果。代价是 owner 的最终退出要等待 child，因此这必须是明确的所有权决定。

不要把整个工具调用链随手套进 `shield()`。那会让用户已经停止时工具仍在运行，而系统又没有清楚的回收者。若说不清谁保存引用、谁等待结束、程序关闭时怎么办，就不应使用它。

21.12 示例运行导览

`example.py` 的完整入口按以下顺序运行：

1. **正常完成**：三层链路得到结果，I/O `finally` 释放一次资源；
2. **用户主动停止**：观察 cancel 返回、异常注入、清理中和最终 cancelled 四阶段；
3. **直接 await 子 Task**：父取消传播给子，父等待子清理后再完成；
4. **异常层级与吞取消反例**：验证 `except Exception` 不捕获取消，以及宽捕获后继续的后果；
5. **`timeout()`**：范围内是 `CancelledError`、范围外是 `TimeoutError`，外部取消不被改写；
6. **`wait_for()`**：单次尝试超时取消目标，并等待人为延迟的清理；
7. **绝对 deadline**：分别触发更短的工具预算和整个请求截止时间；
8. **非硬中断**：同步阻塞完成后，事件循环才处理已经到期的 timeout；
9. **选学 `shield()`**：调用方取消，child 继续，由所有者保存并最终收集。

所有并发顺序断言都先用 `Event` 确认状态，不假设 Task 创建后已经开始，也不依赖多个 Task 的固定调度顺序。短时间值只用于触发超时或展示有限阻塞，没有断言精确耗时。反例最终都会释放资源并收集 Task，不会挂住入口或污染后续段落。

21.13 常见错误与修复方向

错误	可观察后果	修复方向
`task.cancel()` 后立即关闭共享资源	Task 的 `finally` 可能仍在使用或清理资源	由所有者 `await task` 后再越过生命周期边界

21.13 常见错误与修复方向

错误	可观察后果	修复方向
`task.cancel()` 后立即关闭共享资源	Task 的 `finally` 可能仍在使用或清理资源	由所有者 `await task` 后再越过生命周期边界
把 `cancel()` 返回 `True` 当作已经停止	`done()` / `cancelled()` 仍为 `False`	区分请求接受与最终状态
`except BaseException`, 后返回错误字典	用户取消被伪装成业务失败	只为清理或记录时裸 `raise`
`except CancelledError` 后继续下一工具	用户停止后仍产生远程副作用	清理后传播到编排边界
在 timeout 范围内部捕获其 `TimeoutError`	捕获位置看不到最终转换	在 `async with` 外捕获
把任意 `CancelledError` 改成 `TimeoutError`	用户停止被误报成系统超时	让 timeout 管理器只转换自己发起的取消
每层都重新 `timeout(10)`	重试和多工具令总时长膨胀	外层计算一次 `loop.time()` deadline 并向内传递
认为 `wait_for(..., 1)` 一秒后必返回	目标清理或吞取消会延后甚至改变结果	把 1 秒理解为发起取消的等待预算
只保存后台 Task 引用，从不 await	无法确认清理，也可能漏掉异常	在所有者关闭边界取消并收集
用 `shield()` 让所有工作“不受取消”	调用方退出后工作仍游离运行	只屏蔽明确对象，并指定最终回收者

21.14 递进练习

以下练习不给完整答案。建议先写出预测事件，再运行并用断言验证。

练习 1：实现可观察的取消四阶段

练习 1：实现可观察的取消四阶段

要求：实现 `download_model()`，内部打开一个本地 `FakeConnection`，等待 `data_ready`，并在 `finally` 等待 `cleanup_allowed` 后关闭。调用方先等 `started`，再取消 Task。

输入：任务名固定为 `"model-download"`，取消消息固定为 `"request closed"`。

验收标准：

- `cancel()` 刚返回时 `done()` 和 `cancelled()` 都为 `False`；
- `cleanup_started` 后、打开清理闸门前，连接仍打开且 Task 未完成；
- `await task` 得到 `CancelledError`，消息可观察；
- 最终连接只关闭一次，Task 的 `done()` 和 `cancelled()` 都为 `True`。

练习 2：修复“用户已停止但工具仍继续”

要求：给出一个捕获 `BaseException` 后继续执行 `write_result()` 的错误工具链，先复现问题，再把它改为只记录并传播取消。两个版本都必须有限结束并清理资源。

输入：`tool_started`、`write_allowed` 两个 Event；写入内容固定为 `"answer=42"`。

验收标准：

- 错误版能观察到取消后仍写入，并最终错误地 `cancelled() is False`；
- 修复版不调用 `write_result()`，调用方收到 `CancelledError`；
- 两版资源最终都关闭，不遗留 Task；
- 能解释为什么改成 `except Exception` 与专门捕获后裸 `raise` 的适用边界不同。

练习 3：三层预算与重试

要求：整个请求 deadline 为 2 秒，某工具总预算为 1.2 秒，每次尝试预算为 0.4 秒，最多三次。用 `loop.time()` 只计算一次请求 deadline，每次尝试前从工具 deadline 重新计算剩余时间。

输入：本地替身前两次等待到单次尝试超时，第三次由 `success_allowed` Event 决定是否成功；不要访问真实网络。

验收标准：

- 内层收到的是同一个绝对请求 deadline，而不是新的 2 秒；
- 每次尝试的相对 timeout 不超过工具和请求的剩余预算；
- 工具预算耗尽后不再开始新尝试；

验收标准：

- 内层收到的是同一个绝对请求 deadline，而不是新的 2 秒；
- 每次尝试的相对 timeout 不超过工具和请求的剩余预算；
- 工具预算耗尽后不再开始新尝试；
- 取消的尝试都完成 `finally`，断言不依赖恰好在某毫秒返回。

练习 4：比较范围超时与目标超时

要求：分别用 `asyncio.timeout()` 包住"加载 + 解析"两步，以及用 `wait_for()` 只限制加载 Task。让加载 Task 的清理等待一个 Event。

输入：加载永不正常完成；解析函数只记录是否被调用；超时值使用很短但有限的常量。

验收标准：

- `timeout()` 内部取消路径观察到 `CancelledError`，范围外捕获内置 `TimeoutError`；
- `wait_for()` 到期后，在清理 Event 放行前仍未完成；
- 两种路径都不调用解析函数；
- 外部主动取消长 timeout 范围时仍观察到 `CancelledError`。

练习 5（选学）：给 `shield()` 指定真正的所有者

要求：创建一个必须写完本地审计记录的 child Task，owner 用 `shield()` 等待。取消 owner 后，child 由 Event 放行并正常结束，owner 收集 child 后继续传播自己的取消。

输入：审计记录固定为 `"request-aborted"`。

验收标准：

- owner 取消后立刻观察到 `CancelledError`，child 没有因这次取消而变为 cancelled；
- owner 始终保存 child 强引用；
- child 的结果或异常被真正 await 和收集；
- owner 最终仍是 cancelled，不把 `shield()` 当成吞取消工具。

21.15 本章验收、验证与边界

本章文件：

文件	内容

21.15 本章验收、验证与边界

本章文件：

文件	内容
`README.md`	取消协议、传播、超时选择、共同 deadline、边界与练习
`example.py`	九组本地实验、中文事件轨迹和运行时断言

完整入口在 CPython 3.14.7 上执行到末尾，运行时断言验证了：

- 正常完成和用户取消都只释放一次资源；
- `cancel()` 返回时 Task 尚未停止，异步 `finally` 完成前也尚未进入最终取消状态；
- 直接等待子 Task 时，父取消向子传播，并等待子清理；
- `CancelledError` 的继承层级以及吞掉取消后 Task 错误成功的结果；
- `timeout()` 的范围内外异常类型，以及未到期范围不改写普通外部取消；
- `wait_for()` 在目标清理期间仍未完成；
- 工具局部预算和 Agent 请求共同 deadline 的不同边界；
- 同步阻塞使超时回调延后；
- `shield()` 下 child 和调用方各自的最终状态。

同时执行了 Ruff 常规/格式检查和 Pyright 基础类型检查。静态检查验证的是格式、常见缺陷与类型一致性，不等于运行验证。

未验证的范围包括真实 HTTP、LLM、MCP、数据库、线程、操作系统 I/O 取消、连接池、服务端 deadline、事务回滚、重试退避以及远程副作用撤销。本章也不把 `uncancel()` 当作常规修复方案；吞掉取消会破坏 `timeout()` 等依赖取消协议的组件，业务代码应优先保持传播。

学完后，请尝试不看正文逐层回答：

- `cancel()` 返回、Task 收到 `CancelledError`、`finally` 完成和 `cancelled()` 为真分别在何时？
- 为什么 `except Exception` 不捕获取消，而裸 `except` 会？记录取消后为什么通常要 `raise`？
- 父 Task 直接 await 子 Task 时，父取消沿哪条边传播？保存引用解决了什么，又没有解决什么？
- `timeout()` 为什么在范围内表现为取消、在范围外表现为超时？外部取消为何不能随意改写？
- `wait_for()` 与 `timeout()` 分别限制目标还是范围，为什么两者都可能晚于数字本身返回？
- 为什么共同 deadline 必须用 `loop.time()` 只计算一次，并向内层传递绝对值？
- 本地 Task 已取消，为什么线程、服务器或已经提交的副作用仍可能继续？

```python
"""第 21 章：取消、超时与截止时间。

示例只使用本地 Event 和可观察资源状态，不访问真实网络或线程。短超时只负责
触发 asyncio 的超时机制；断言验证事件顺序和最终状态，不断言精确耗时。
"""

# 建议按 run_examples() 中的编号顺序阅读：先看正常完成的基线，再逐步加入主动取消、
# 相对超时、绝对截止时间和 shield。所有实验都把关键动作写入 events，最后用断言验证
# “先发生什么、后发生什么”；短小的超时数值只负责触发机制，不是要测事件循环性能。
# asyncio 提供 Task、Event 和超时上下文；time 只在“阻塞事件循环”的反例中使用。
import asyncio
import time

# Awaitable[T] 表示“可被 await、完成后产生 T”的对象，既可描述 coroutine，也可描述 Task。
from collections.abc import Awaitable

# Enum 建模资源的离散状态；auto() 自动分配成员值，本例只关心成员身份而不关心具体数字。
from enum import Enum, auto


# 三个枚举成员描述 FakeIOResource 从创建到释放的单向生命周期。
# Enum 成员是单例，因此后面用 is / is not 比较成员身份，而不是比较字符串或整数值。
class ResourceState(Enum):
    """模拟 I/O 资源的最小生命周期。"""

    NEW = auto()
    OPEN = auto()
    RELEASED = auto()


# 自定义异常不需要额外方法：继承 TimeoutError 后，既能被精确捕获为“工具预算耗尽”，
# 也保留了超时异常的类别关系。类体中的 docstring 已足以构成合法类体。
class ToolBudgetExceeded(TimeoutError):
    """表示工具自己的预算先于请求总截止时间耗尽。"""


# 标既便码子数设有只指示作 ... 表示实际产生输出，无返回有意义的业务值。
```

```
36
37  # 标题打印函数没有异步工作；-> None 表示它只产生输出，不返回有意义的业务值。
38  def print_section(title: str) -> None:
39      print(f"\n=== {title} ===")
40
41
42  # events 是调用方传入的同一个可变列表。append() 会原地写入共享的事件轨迹，print() 则让
43  # 运行者同步看到轨迹；后面的 index() 和生成器表达式会基于这份列表验证因果顺序。
44  def record(events: list[str], message: str) -> None:
45      """事件同时打印和保存，便于肉眼观察并做因果顺序断言。"""
46      events.append(message)
47      print(f"  事件 | {message}")
48
49
50  class FakeIOResource:
51      """只模拟本地资源状态，不声称等价于 socket、HTTP 或远程服务。"""
52
53      def __init__(self, name: str, events: list[str]) -> None:
54          # self 保存每个资源实例自己的属性；events 不会被复制，因此资源层和调用层记录到
55          # 同一个列表中。初始状态 NEW 与计数 0 为后面的"一次且仅一次释放"断言提供基线。
56          self.name: str = name
57          self.events: list[str] = events
58          self.state: Literal[ResourceState.NEW] = ResourceState.NEW
59          self.release_count = 0
60
61      def open(self) -> None:
62          # 非 NEW 状态意味着资源已经打开过或释放过。先检查再改状态，能让生命周期错误立即
63          # 暴露，而不是悄悄覆盖旧状态。
64          if self.state is not ResourceState.NEW:
65              raise RuntimeError(f"{self.name}: 资源只能打开一次")
66          self.state: Literal[ResourceState.OPEN] = ResourceState.OPEN
67          record(self.events, f"{self.name}: 资源已打开")
68
69      def release(self) -> None:
70          # 本例把重复释放视为错误，借此验证 finally 没有运行两次。真实资源若要求幂等关闭，
71          # 可以采用不同策略，但仍应明确记录所有权。
72          if self.state is ResourceState.RELEASED:
```

```
50  class FakeIOResource:
69      def release(self) -> None:
70          # 本例把重复释放视为错误，借此验证 finally 没有运行两次。真实资源若要求幂等关闭，
71          # 可以采用不同策略，但仍应明确记录所有权。
72          if self.state is ResourceState.RELEASED:
73              raise RuntimeError(f"{self.name}: 资源被重复释放")
74          self.state: Literal[ResourceState.RELEASED] = ResourceState.RELEASED
75          self.release_count += 1
76          record(self.events, f"{self.name}: 资源已释放")
77
78
79  async def simulated_io(
80      resource: FakeIOResource,
81      work_started: asyncio.Event,
82      work_may_finish: asyncio.Event,
83      cleanup_started: asyncio.Event,
84      cleanup_may_finish: asyncio.Event | None = None,
85  ) -> str:
86      """三层链路的 I/O 层：取消时先记录，再由 finally 释放资源。"""
87      # 四个 Event 都是本地"闸门"：set() 只把状态改为已设置并唤醒等待者，wait() 则在未设置
88      # 时挂起当前 Task。用它们协调实验，可准确知道工作和清理走到了哪里，不必靠 sleep 猜测。
89      resource.open()
90      try:
91          # 先记录并通知调用方"I/O 已就位"，随后等待 work_may_finish。正常实验会打开这个
92          # 闸门；取消实验保持它关闭，使 Task 稳定地停在这个 await 上等待取消注入。
93          record(resource.events, f"{resource.name}: I/O 已进入等待阶段")
94          work_started.set()
95          await work_may_finish.wait()
96          record(resource.events, f"{resource.name}: I/O 正常完成")
97          return f"{resource.name} 的结果"
98      except asyncio.CancelledError as error:
99          # cancel(message) 传入的消息可出现在 error.args 中；!r 使用 repr 形式显示参数元组，
100         # 便于区分空参数 () 和带消息的 ('用户按下停止',)。这里只为记录传播方向。
101         # 裸 raise 会原样重抛当前异常；若在此返回普通值，Task 会被标记为成功而不是取消。
102         record(resource.events, f"{resource.name}: I/O 层收到 CancelledError{error.args!r}")
103         raise
```

```python
async def simulated_io(
    finally:
        # finally 在正常返回和异常传播两条路径都会执行，因此资源释放只需放在一个位置。
        # cleanup_started 让外层观察“取消已经注入、但资源还没释放”的中间阶段。
        record(resource.events, f"{resource.name}: finally 开始清理")
        cleanup_started.set()
        if cleanup_may_finish is not None:
            # 某些真实清理也是异步的。任务在这段时间仍未完成，也尚未进入
            # cancelled() 为真的最终状态。is not None 是在判断是否提供了可选闸门；
            # Event 本身不应拿来做真假值判断。
            await cleanup_may_finish.wait()
        resource.release()


async def call_tool(
    resource: FakeIOResource,
    work_started: asyncio.Event,
    work_may_finish: asyncio.Event,
    cleanup_started: asyncio.Event,
    cleanup_may_finish: asyncio.Event | None = None,
) -> str:
    """工具层直接 await I/O；取消沿同一个等待链向内传播。"""
    # 这里没有 create_task()，所以 call_tool 与 simulated_io 属于同一个 Task 的协程调用栈。
    # result 只有在 I/O 正常返回时才会绑定；取消会直接跳到 except 分支。
    record(resource.events, f"{resource.name}: 工具层开始")
    try:
        result: str = await simulated_io(
            resource,
            work_started,
            work_may_finish,
            cleanup_started,
            cleanup_may_finish,
        )
        record(resource.events, f"{resource.name}: 工具层得到结果")
        return result
    except asyncio.CancelledError:
```

```
117 async def call_tool(
138     except asyncio.CancelledError:
139         # 工具层可以记录自己的语义上下文，但仍用裸 raise 保留异常类型、消息和 traceback。
140         record(resource.events, f"{resource.name}: 工具层继续传播取消")
141         raise
142
143
144 async def request_entry(
145     resource: FakeIOResource,
146     work_started: asyncio.Event,
147     work_may_finish: asyncio.Event,
148     cleanup_started: asyncio.Event,
149     cleanup_may_finish: asyncio.Event | None = None,
150 ) -> str:
151     """入口层不把取消伪装成普通业务失败。"""
152     # 入口 → 工具 → I/O 三层连续 await。任何一层收到取消后原样抛出，最终拥有 Task 的
153     # 调用方才能区分"用户停止"和普通业务错误。
154     record(resource.events, f"{resource.name}: 入口层开始")
155     try:
156         return await call_tool(
157             resource,
158             work_started,
159             work_may_finish,
160             cleanup_started,
161             cleanup_may_finish,
162         )
163     except asyncio.CancelledError:
164         record(resource.events, f"{resource.name}: 入口层继续传播取消")
165         raise
166
167
168 async def demonstrate_normal_completion() -> None:
169     # 正常路径是后续实验的对照组：工作闸门会被主动放行，没有任何 cancel 或 timeout。
170     print_section("1. 正常完成：入口 → 工具 → 模拟 I/O → finally")
171     # 每个实验创建独立的列表、资源和 Event，避免上一段的状态污染下一段。
172     events: list[str] = []
```

```python
async def demonstrate_normal_completion() -> None:
    # 正常路径是后续实验的对照组：工作闸门会被主动放行，没有任何 cancel 或 timeout。
    print_section("1. 正常完成：入口 → 工具 → 模拟 I/O → finally")
    # 每个实验创建独立的列表、资源和 Event，避免上一段的状态污染下一段。
    events: list[str] = []
    resource = FakeIOResource("normal", events)
    work_started = asyncio.Event()
    work_may_finish = asyncio.Event()
    cleanup_started = asyncio.Event()

    # 调用 async def 只会得到 coroutine 对象；create_task() 才把它交给事件循环并立即返回
    # 可管理的 Task。name 只用于诊断，不影响调度。
    task: Task[str] = asyncio.create_task(
        request_entry(resource, work_started, work_may_finish, cleanup_started),
        name="normal-tool-chain",
    )
    # 等 I/O 层明确报告已经开始，再打开完成闸门；set() 后所有当前及未来 wait() 都可通过。
    await work_started.wait()
    work_may_finish.set()
    # await Task 取得 request_entry() 最终返回的字符串，并保证其 finally 已经执行完毕。
    result: str = await task

    # 输出展示返回值、Task 终态和资源终态；断言把教学预期变成可执行验收条件。
    print("  调用结果:", result)
    print("  Task 状态: done=", task.done(), "cancelled=", task.cancelled())
    print("  资源状态:", resource.state.name)
    assert cleanup_started.is_set()
    assert resource.state is ResourceState.RELEASED
    assert resource.release_count == 1
    assert task.done() and not task.cancelled()


async def demonstrate_cancellation_phases() -> None:
    # 这一段故意把异步清理暂停，分离“cancel 请求已接受”“异常已注入”“清理中”和
    # “Task 最终 cancelled”四个容易混淆的时刻。
    print_section("2. 用户主动停止：取消请求与最终取消是不同阶段")
```

```python
async def demonstrate_cancellation_phases() -> None:
    # 这一段故意把异步清理暂停，分离“cancel 请求已接受”“异常已注入”“清理中”和
    # “Task 最终 cancelled”四个容易混淆的时刻。
    print_section("2. 用户主动停止：取消请求与最终取消是不同阶段")
    events: list[str] = []
    resource = FakeIOResource("user-stop", events)
    work_started = asyncio.Event()
    work_may_finish = asyncio.Event()
    cleanup_started = asyncio.Event()
    cleanup_may_finish = asyncio.Event()

    # 额外传入 cleanup_may_finish，使 simulated_io 的 finally 可以被测试代码控制。
    task: Task[str] = asyncio.create_task(
        request_entry(
            resource,
            work_started,
            work_may_finish,
            cleanup_started,
            cleanup_may_finish,
        ),
        name="user-stoppable-chain",
    )

    # Event 证明 I/O 已经进入等待阶段，取消不依赖“猜它大概启动了”。cancel() 的字符串
    # 会随 CancelledError 传播；布尔返回值只说明本次取消请求能否被登记。
    await work_started.wait()
    cancellation_accepted: bool = task.cancel("用户按下停止")
    record(events, "调用方: cancel() 已返回")

    # cancel() 只登记请求；当前代码尚未把执行权交回循环，目标 Task 还没机会
    # 在 await 位置收到 CancelledError。done() 表示是否已有最终结果/异常/取消状态，
    # cancelled() 只在 coroutine 最终以 CancelledError 结束后才为 True。
    print("  cancel() 接受请求:", cancellation_accepted)
    print("  刚返回时: done=", task.done(), "cancelled=", task.cancelled())
    assert cancellation_accepted
    assert not task.done()
```

```
200  async def demonstrate_cancellation_phases() -> None:
233      print("  刚返回时: done=", task.done(), "cancelled=", task.cancelled())
234      assert cancellation_accepted
235      assert not task.done()
236      assert not task.cancelled()
237      assert not cleanup_started.is_set()
238      assert not any("I/O 层收到 CancelledError" in event for event in events)
239
240      # await cleanup_started.wait() 会把执行权交还事件循环。目标 Task 因而有机会在
241      # work_may_finish.wait() 处收到取消，逐层传播并进入 finally；清理闸门此时仍关闭。
242      await cleanup_started.wait()
243      print("  finally 进行中: done=", task.done(), "cancelled=", task.cancelled())
244      print("  finally 进行中资源仍为:", resource.state.name)
245      assert resource.state is ResourceState.OPEN
246      assert not task.done()
247      assert not task.cancelled()
248
249      # 放行清理后再 await task，既收集取消，也与资源释放完成建立明确的先后关系。
250      cleanup_may_finish.set()
251      try:
252          await task
253      except asyncio.CancelledError as error:
254          # 此处是 Task 所有者的取消边界：它确认取消和 finally 都已完成，不再重新抛给
255          # run_examples()，这样后续教学实验还能继续运行。
256          record(events, f"调用方: await Task 收到 CancelledError{error.args!r}")
257      else:
258          # try 的 else 仅在 await 没有抛异常时执行；走到这里说明取消被错误地吞掉了。
259          raise AssertionError("正确实现不应把用户取消伪装成成功")
260
261      print("  清理后: done=", task.done(), "cancelled=", task.cancelled())
262      print("  清理后资源:", resource.state.name)
263      assert resource.state is ResourceState.RELEASED
264      assert resource.release_count == 1
265      assert task.done() and task.cancelled()
266
267      # index() 返回事件第一次出现的位置；next(生成器) 则取第一个满足文本条件的位置。
```

```
200 async def demonstrate_cancellation_phases() -> None:
267     # index() 返回事件第一次出现的位置；next(生成器) 则取第一个满足文本条件的位置。
268     # 最后只比较相对大小，从而验证因果链，而不依赖具体调度时间或事件列表的绝对长度。
269     cancel_returned: int = events.index("调用方: cancel() 已返回")
270     cancellation_injected: int = next(
271         index for index, event in enumerate(events) if "I/O 层收到 CancelledError" in event
272     )
273     cleanup_finished: int = events.index("user-stop: 资源已释放")
274     caller_observed: int = next(
275         index for index, event in enumerate(events) if event.startswith("调用方: await Task")
276     )
277     assert cancel_returned < cancellation_injected < cleanup_finished < caller_observed
278
279
280 async def parent_directly_awaits_task(
281     child: asyncio.Task[str],
282     parent_is_waiting: asyncio.Event,
283     events: list[str],
284 ) -> str:
285     # parent_is_waiting 在直接 await 之前置位，让外层可以确定父 Task 已进入这条等待边。
286     # 父 Task 若被取消，取消会传递给它当前正在等待的 child Task。
287     record(events, "parent: 即将直接 await child Task")
288     parent_is_waiting.set()
289     try:
290         return await child
291     except asyncio.CancelledError:
292         record(events, "parent: child 清理结束后收到并传播 CancelledError")
293         raise
294
295
296 async def demonstrate_direct_await_propagation() -> None:
297     # 这里显式创建两个 Task，专门观察取消如何沿“父 Task 正在 await 子 Task”这条边传播。
298     print_section("3. 取消正在直接 await 子 Task 的父 Task")
299     events: list[str] = []
300     resource = FakeIOResource("child", events)
301     work_started = asyncio.Event()
```

```python
async def demonstrate_direct_await_propagation() -> None:
    # 这里显式创建两个 Task，专门观察取消如何沿“父 Task 正在 await 子 Task”这条边传播。
    print_section("3. 取消正在直接 await 子 Task 的父 Task")
    events: list[str] = []
    resource = FakeIOResource("child", events)
    work_started = asyncio.Event()
    work_may_finish = asyncio.Event()
    cleanup_started = asyncio.Event()
    cleanup_may_finish = asyncio.Event()
    parent_is_waiting = asyncio.Event()

    # 保存引用让所有者可以观察和最终收集子 Task；仅保存并不等于清理已结束。
    # child 先启动并停在 I/O 闸门，确保随后创建 parent 时等待的是一个未完成 Task。
    child_task: Task[str] = asyncio.create_task(
        call_tool(
            resource,
            work_started,
            work_may_finish,
            cleanup_started,
            cleanup_may_finish,
        ),
        name="directly-awaited-child",
    )
    await work_started.wait()
    # parent_is_waiting 消除调度猜测：只有父协程执行到 child 的直接 await 前才继续实验。
    parent_task: Task[str] = asyncio.create_task(
        parent_directly_awaits_task(child_task, parent_is_waiting, events),
        name="parent-awaiting-child",
    )
    await parent_is_waiting.wait()

    # 父 Task 的 cancel() 会调用它当前所等待 child 的取消逻辑。cancelling() 返回待处理的
    # 取消请求数；大于 0 证明请求已经到达 child，但不代表 child 的清理已经完成。
    parent_task.cancel("停止父任务")
    assert child_task.cancelling() > 0
    await cleanup_started.wait()
```

```
296  async def demonstrate_direct_await_propagation() -> None:
332
333      print("  子 Task 引用仍在，但清理尚未结束:", not child_task.done())
334      print("  父 Task 也在等待传播完成:", not parent_task.done())
335      assert resource.state is ResourceState.OPEN
336      assert not child_task.done()
337      assert not parent_task.done()
338
339      # child 的 finally 被放行后，child 先结束取消；父 Task 的 await 随后抛出取消并继续向外。
340      cleanup_may_finish.set()
341      try:
342          await parent_task
343      except asyncio.CancelledError:
344          # 等待 parent 等价于间接等完 child 的这条传播链；调用方在这里收集最终取消。
345          record(events, "调用方: 已等待父 Task 完成取消")
346      else:
347          raise AssertionError("父 Task 的取消不应消失")
348
349      # 直接 await 形成传播链：父 Task 被取消时，正在等待的 child 也收到取消。
350      # 父任务完成前已经等到了 child 的异步 finally 结束。
351      print("  child: done=", child_task.done(), "cancelled=", child_task.cancelled())
352      print("  parent: done=", parent_task.done(), "cancelled=", parent_task.cancelled())
353      assert child_task.done() and child_task.cancelled()
354      assert parent_task.done() and parent_task.cancelled()
355      assert resource.state is ResourceState.RELEASED
356
357
358  async def catches_exception_only(started: asyncio.Event, events: list[str]) -> None:
359      """证明 except Exception 不会拦截 CancelledError。"""
360      # 新建且永不 set 的 Event 让任务稳定挂起，直到外部取消。CancelledError 直接继承
361      # BaseException，不属于 Exception，因此 except 分支不执行，但 finally 仍会执行。
362      try:
363          started.set()
364          await asyncio.Event().wait()
365      except Exception:  # noqa: BLE001 - 本函数专门验证它不会捕获取消。
366          record(events, "错误: except Exception 捕获了取消")
367      finally:
```

```python
async def catches_exception_only(started: asyncio.Event, events: list[str]) -> None:
    """证明 except Exception 不会拦截 CancelledError。"""
    # 新建且永不 set 的 Event 让任务稳定挂起，直到外部取消。CancelledError 直接继承
    # BaseException，不属于 Exception，因此 except 分支不执行，但 finally 仍会执行。
    try:
        started.set()
        await asyncio.Event().wait()
    except Exception:  # noqa: BLE001 - 本函数专门验证它不会捕获取消。
        record(events, "错误: except Exception 捕获了取消")
    finally:
        record(events, "except Exception 示例: finally 已运行")


async def swallowing_tool(
    resource: FakeIOResource,
    work_started: asyncio.Event,
    continuation_allowed: asyncio.Event,
    cancellation_swallowed: asyncio.Event,
) -> str:
    """教学反例：捕获 BaseException 后继续，令已停止的工具仍产生副作用。"""
    # 这是刻意保留的错误写法：BaseException 连取消和进程退出类异常也会捕获；捕获后又
    # 没有 raise，导致控制流落到后续副作用。外层用 Event 限制它，避免示例永久挂起。
    resource.open()
    try:
        # 告知调用方可以安全发出 cancel，然后在一个永不自行完成的 wait() 上接收取消。
        work_started.set()
        record(resource.events, "反例工具: 等待第一次工作")
        try:
            await asyncio.Event().wait()
        except BaseException as error:  # noqa: BLE001 - 故意展示危险反例。
            # type(error).__name__ 只记录实际异常类名。这里 set() 表示“异常已被吞”，
            # 并非表示 Task 已完成；执行会继续越过 except。
            record(resource.events, f"反例工具: 错误地吞掉 {type(error).__name__}")
            cancellation_swallowed.set()

        # continuation_allowed 模拟调用方后来放行一项有副作用的操作。正确代码重新抛出
```

```python
async def swallowing_tool(
        # continuation_allowed 模拟调用方后来放行一项有副作用的操作。正确代码重新抛出
        # CancelledError 后不会到达这里。
        record(resource.events, "反例工具: 取消后仍继续")
        await continuation_allowed.wait()
        record(resource.events, "反例工具: 取消后仍提交了副作用")
        return "伪成功"
    finally:
        # 即便错误地吞了取消，finally 仍会在普通 return 前执行，所以资源仍能释放；
        # “资源已释放”并不能证明取消语义正确。
        resource.release()


async def demonstrate_exception_hierarchy_and_swallowing() -> None:
    # 先用运行时类型关系验证异常层级，再运行一个不会吞取消的边界和一个会吞取消的反例。
    print_section("4. CancelledError 层级与吞掉取消的受控反例")
    # __mro__ 是方法解析顺序元组：索引 0 是类本身，索引 1 是直接父类。
    direct_parent: type = asyncio.CancelledError.__mro__[1]
    print("  CancelledError 的直接父类:", direct_parent.__name__)
    assert issubclass(asyncio.CancelledError, BaseException)
    assert not issubclass(asyncio.CancelledError, Exception)

    # 第一小段取消 catches_exception_only。若 except Exception 错误地捕获取消，事件列表会
    # 出现以“错误:”开头的记录，最后的 any(...) 断言就会失败。
    exception_events: list[str] = []
    exception_started = asyncio.Event()
    exception_task: Task[None] = asyncio.create_task(
        catches_exception_only(exception_started, exception_events),
        name="except-exception-demo",
    )
    await exception_started.wait()
    exception_task.cancel()
    try:
        await exception_task
    except asyncio.CancelledError:
        # 取消越过 except Exception 到达 Task 所有者，证明它没有被当作普通业务异常处理。
```

```
405  async def demonstrate_exception_hierarchy_and_swallowing() -> None:
424      try:
425          await exception_task
426      except asyncio.CancelledError:
427          # 取消越过 except Exception 到达 Task 所有者，证明它没有被当作普通业务异常处理。
428          record(exception_events, "调用方: except Exception 未拦截取消")
429      assert not any(event.startswith("错误:") for event in exception_events)
430
431      # 第二小段使用独立状态复现吞取消。cancellation_swallowed 只通知测试代码反例已越过
432      # 错误捕获点，continuation_allowed 则控制后续副作用何时发生。
433      bad_events: list[str] = []
434      bad_resource = FakeIOResource("swallow", bad_events)
435      work_started = asyncio.Event()
436      continuation_allowed = asyncio.Event()
437      cancellation_swallowed = asyncio.Event()
438      bad_task: Task[str] = asyncio.create_task(
439          swallowing_tool(
440              bad_resource,
441              work_started,
442              continuation_allowed,
443              cancellation_swallowed,
444          ),
445          name="bad-cancellation-swallowing",
446      )
447      await work_started.wait()
448      bad_task.cancel("用户早已停止")
449      await cancellation_swallowed.wait()
450
451      # Task 仍在等待 continuation_allowed，因此既没成功也没进入最终取消状态。
452      print("  取消已被吞掉，Task 仍未结束:", not bad_task.done())
453      assert not bad_task.done()
454      # 放行后反例会返回普通字符串；await 因而拿到“伪成功”，不会抛 CancelledError。
455      continuation_allowed.set()
456      result: str = await bad_task
457
458      # cancelled() 为 False 说明 Task 的最终状态是成功完成。事件与资源断言分别证明
```

```
405 async def demonstrate_exception_hierarchy_and_swallowing() -> None:
457
458     # cancelled() 为 False 说明 Task 的最终状态是成功完成。事件与资源断言分别证明
459     # 错误副作用确实发生、finally 仍执行，这两件事必须分开判断。
460     print("  反例返回:", result)
461     print("  反例最终 cancelled():", bad_task.cancelled())
462     assert result == "伪成功"
463     assert not bad_task.cancelled()
464     assert "反例工具: 取消后仍提交了副作用" in bad_events
465     assert bad_resource.state is ResourceState.RELEASED
466
467
468 async def timeout_scope(
469     resource: FakeIOResource,
470     work_started: asyncio.Event,
471     cleanup_started: asyncio.Event,
472     timeout_seconds: float,
473 ) -> None:
474     """在 timeout 范围内观察 CancelledError，在范围外观察 TimeoutError。"""
475     # 这个 Event 永不由正常路径 set，因此 request_entry 会一直等待，直到超时机制取消
476     # 执行 timeout_scope 的“当前 Task”。timeout() 不会为代码块另建 worker Task。
477     work_may_finish = asyncio.Event()
478     # 保存进入范围前的 Task 身份，用 is 验证进入异步上下文后仍在同一个 Task 中执行。
479     current_before: Task[Any] | None = asyncio.current_task()
480     try:
481         # timeout_seconds 是相对秒数。到期后上下文管理器先向当前 Task 发出取消请求。
482         async with asyncio.timeout(timeout_seconds):
483             assert asyncio.current_task() is current_before
484             try:
485                 await request_entry(resource, work_started, work_may_finish, cleanup_started)
486             except asyncio.CancelledError:
487                 # 范围内部先看到取消。必须重新抛出，外层 timeout 管理器退出时才能识别
488                 # 这是自己触发的取消，并将它转换为内置 TimeoutError。
489                 record(resource.events, "timeout 范围内: 看到 CancelledError，继续抛出")
490                 raise
491     except TimeoutError:
492         # 只有离开 async with 后才会看到该上下文转换出的 TimeoutError。
```

```
async def timeout_scope(

    except TimeoutError:
        # 只有离开 async with 后才会看到该上下文转换出的 TimeoutError。
        record(resource.events, "timeout 范围外: 捕获 TimeoutError")
    else:
        # try 的 else 代表范围没有抛出 TimeoutError；对永不完成的工作来说这是协议错误。
        raise AssertionError("未完成的 I/O 应触发范围超时")


async def external_cancel_inside_timeout(
    resource: FakeIOResource,
    work_started: asyncio.Event,
    cleanup_started: asyncio.Event,
) -> None:
    """长超时未到期时，外部 cancel 不能被上下文改写成 TimeoutError。"""
    # 60 秒远长于本地实验时间，因此稍后发生的取消来自 wrapper_task.cancel()，不是这个
    # timeout 范围。传入的新 Event 不会被 set，使 I/O 保持在可取消的等待点。
    try:
        async with asyncio.timeout(60):
            await request_entry(
                resource,
                work_started,
                asyncio.Event(),
                cleanup_started,
            )
    except TimeoutError:
        # 若走到这里，说明外部用户取消被错误地解释为该范围自己的超时。
        record(resource.events, "错误: 外部取消被误报成超时")
        raise


async def demonstrate_timeout_context() -> None:
    # 第一小段触发范围自己的 deadline；第二小段在长 timeout 尚未到期时从外部主动取消。
    print_section("5. asyncio.timeout() 取消当前 Task 并在范围外转换异常")
    events: list[str] = []
    resource = FakeIOResource("tool-timeout", events)
```

```python
async def demonstrate_timeout_context() -> None:
    # 第一小段触发范围自己的 deadline；第二小段在长 timeout 尚未到期时从外部主动取消。
    print_section("5. asyncio.timeout() 取消当前 Task 并在范围外转换异常")
    events: list[str] = []
    resource = FakeIOResource("tool-timeout", events)
    work_started = asyncio.Event()
    cleanup_started = asyncio.Event()

    # timeout_scope 自己在范围外收集 TimeoutError，所以这里会正常返回。随后分别验证工作
    # 确实开始过、finally 确实开始并完成，以及异常转换发生在范围内记录之后。
    await timeout_scope(resource, work_started, cleanup_started, timeout_seconds=0.01)
    assert work_started.is_set()
    assert cleanup_started.is_set()
    assert resource.state is ResourceState.RELEASED
    inside_index: int = events.index("timeout 范围内: 看到 CancelledError, 继续抛出")
    outside_index: int = events.index("timeout 范围外: 捕获 TimeoutError")
    assert inside_index < outside_index

    # 为外部取消实验创建独立 Task，才能由当前调用方调用 cancel()。
    external_events: list[str] = []
    external_resource = FakeIOResource("external-cancel", external_events)
    external_started = asyncio.Event()
    external_cleanup_started = asyncio.Event()
    wrapper_task: Task[None] = asyncio.create_task(
        external_cancel_inside_timeout(
            external_resource,
            external_started,
            external_cleanup_started,
        ),
        name="external-cancel-inside-timeout",
    )
    # 等待 I/O 确认已进入长 timeout 范围，再从范围外取消整个 wrapper Task。
    await external_started.wait()
    wrapper_task.cancel("来自 timeout 范围外的停止")
    try:
        await wrapper_task
```

```python
async def demonstrate_timeout_context() -> None:
    try:
        await wrapper_task
    except asyncio.CancelledError:
        # 长 timeout 并未到期，所以它不会把别处发起的 CancelledError 改写为 TimeoutError。
        record(external_events, "调用方：外部取消仍是 CancelledError")
    else:
        raise AssertionError("普通外部取消不应变成 TimeoutError")

    assert not any(event.startswith("错误:") for event in external_events)
    assert external_resource.state is ResourceState.RELEASED


async def wait_for_before_deadline[T](
    awaitable: Awaitable[T],
    *,
    attempt_budget: float,
    absolute_deadline: float,
) -> T:
    """单次尝试不能拿到超过上层绝对截止时间的相对预算。"""
    # [T] 声明泛型类型参数：输入 awaitable 完成后产生 T，本函数也返回同一种 T。
    # 参数列表中的 * 让后两个预算只能按名字传入，避免两个 float 因位置写反。
    loop: AbstractEventLoop = asyncio.get_running_loop()
    # loop.time() 是单调时钟。deadline 已过时差为负，用 max(..., 0.0) 截成零；再取
    # attempt_budget 与剩余总预算的较小值，保证本次等待不会重新获得一整份上层预算。
    remaining: float = max(0.0, absolute_deadline - loop.time())
    # wait_for 约束传入的明确 awaitable。到期时它会向目标发出取消，并继续等目标完成清理；
    # 本例目标会传播取消，所以等待边界随后得到 TimeoutError。
    return await asyncio.wait_for(awaitable, timeout=min(attempt_budget, remaining))


async def demonstrate_wait_for_cleanup() -> None:
    # 这里把“被限制的 target”和“执行 wait_for 的 waiter”都做成 Task，以便同时观察状态。
    print_section("6. wait_for() 取消目标，并等待目标清理结束")
    events: list[str] = []
    resource = FakeIOResource("wait-for", events)
```

```python
async def demonstrate_wait_for_cleanup() -> None:
    # 这里把“被限制的 target”和“执行 wait_for 的 waiter”都做成 Task，以便同时观察状态。
    print_section("6. wait_for() 取消目标，并等待目标清理结束")
    events: list[str] = []
    resource = FakeIOResource("wait-for", events)
    work_started = asyncio.Event()
    work_may_finish = asyncio.Event()
    cleanup_started = asyncio.Event()
    cleanup_may_finish = asyncio.Event()

    # target 的正常工作和异步清理闸门都保持关闭，使超时后能稳定停在 finally 中。
    target_task: Task[str] = asyncio.create_task(
        call_tool(
            resource,
            work_started,
            work_may_finish,
            cleanup_started,
            cleanup_may_finish,
        ),
        name="wait-for-target",
    )
    # 先等 target 真正进入工作阶段，再创建 waiter，避免 0.01 秒预算在 target 启动前就消耗。
    await work_started.wait()
    loop: AbstractEventLoop = asyncio.get_running_loop()
    # 这里单次尝试只允许 0.01 秒；上层绝对 deadline 尚余约 1 秒，所以 min() 最终采用
    # 更严格的单次预算。waiter_task 代表正在执行 wait_for 的所有者。
    waiter_task: Task[str] = asyncio.create_task(
        wait_for_before_deadline(
            target_task,
            attempt_budget=0.01,
            absolute_deadline=loop.time() + 1.0,
        ),
        name="wait-for-owner",
    )

    # ... [illegible] 但上为设置的
```

```
585 async def demonstrate_wait_for_cleanup() -> None:
619
620     # wait_for 的计时到期后会取消 target；target 已进入 finally，但人为设置的
621     # 清理闸门尚未打开，因此 wait_for 自己也不会提前报告 TimeoutError。
622     await cleanup_started.wait()
623     print("  清理闸门关闭时 target.done():", target_task.done())
624     print("  清理闸门关闭时 wait_for.done():", waiter_task.done())
625     assert not target_task.done()
626     assert not waiter_task.done()
627     assert resource.state is ResourceState.OPEN
628
629     # 放行 target 的 finally 后，wait_for 才能确认目标以取消结束，并向 waiter 报告超时。
630     cleanup_may_finish.set()
631     try:
632         await waiter_task
633     except TimeoutError:
634         # TimeoutError 是 wait_for 对“本次目标超时”的外部表达；target 自身终态仍是取消。
635         record(events, "wait_for 调用方：清理完成后收到 TimeoutError")
636     else:
637         raise AssertionError("单次尝试应已超时")
638
639     # 两个断言分别检查控制流终态与资源终态，证明报告超时时清理已经完成。
640     assert target_task.done() and target_task.cancelled()
641     assert resource.state is ResourceState.RELEASED
642
643
644 async def tool_with_local_budget(
645     resource: FakeIOResource,
646     work_started: asyncio.Event,
647     cleanup_started: asyncio.Event,
648     *,
649     request_deadline: float,
650     tool_budget: float,
651 ) -> str:
652     """只在工具预算比请求剩余预算更严格时增加内层 timeout_at。"""
653     # request_deadline 是外层算好并传进来的绝对时刻；tool_budget 是从“现在”开始计算的
654     # 工具局部相对预算。两者必须都基于当前事件循环的单调时钟。
```

```
644  async def tool_with_local_budget(
652      """只在工具预算比请求剩余预算更严格时增加内层 timeout_at。"""
653      # request_deadline 是外层算好并传进来的绝对时刻；tool_budget 是从“现在”开始计算的
654      # 工具局部相对预算。两者必须都基于当前事件循环的单调时钟。
655      loop: AbstractEventLoop = asyncio.get_running_loop()
656      tool_deadline: float = loop.time() + tool_budget
657      # 正常完成闸门故意保持关闭，让更早的那个 deadline 决定本次调用如何结束。
658      work_may_finish = asyncio.Event()
659
660      # 只有工具自己的截止时刻更早，它才拥有一个独立、可命名的超时边界。
661      if tool_deadline < request_deadline:
662          record(resource.events, "预算: 工具使用更早的绝对截止时间")
663          try:
664              # timeout_at 接受绝对单调时钟时刻；到期时取消执行当前范围的 Task。
665              async with asyncio.timeout_at(tool_deadline):
666                  return await call_tool(
667                      resource,
668                      work_started,
669                      work_may_finish,
670                      cleanup_started,
671                  )
672          except TimeoutError as error:
673              # from error 建立显式异常链：对调用方暴露更具体的工具预算异常，同时保留底层
674              # TimeoutError 作为 __cause__，便于日志和调试追溯。
675              raise ToolBudgetExceeded(f"{resource.name}: 工具预算耗尽") from error
676
677      # 工具没有更严格的局部预算时，只沿用外层请求范围，避免在相同截止时刻
678      # 嵌套两个范围后误把“请求截止”标成“工具截止”。这里的 await 仍会响应外层 Task
679      # 收到的取消，只是不在工具层把它重新解释为 ToolBudgetExceeded。
680      record(resource.events, "预算: 工具沿用请求的绝对截止时间")
681      return await call_tool(resource, work_started, work_may_finish, cleanup_started)
682
683
684  async def demonstrate_absolute_deadlines() -> None:
685      # 两个子实验使用相反的预算大小：第一个由工具局部预算先到期，第二个由请求 deadline
686      # 先到期，从而验证异常应由真正拥有该截止时间的边界命名。
```

```python
async def demonstrate_absolute_deadlines() -> None:
    # 两个子实验使用相反的预算大小：第一个由工具局部预算先到期，第二个由请求 deadline
    # 先到期，从而验证异常应由真正拥有该截止时间的边界命名。
    print_section("7. 单工具预算与整个请求的共同截止时间")
    # 同一事件循环的 time() 值才能彼此比较；它是单调时钟读数，不是 Unix 时间戳。
    loop: AbstractEventLoop = asyncio.get_running_loop()

    # 请求还有约 1 秒，而工具只分到 0.01 秒，所以 tool_with_local_budget 创建内层范围。
    tool_events: list[str] = []
    tool_resource = FakeIOResource("local-budget", tool_events)
    try:
        await tool_with_local_budget(
            tool_resource,
            asyncio.Event(),
            asyncio.Event(),
            request_deadline=loop.time() + 1.0,
            tool_budget=0.01,
        )
    except ToolBudgetExceeded as error:
        # type(...).__name__ 和 str(...) 分别展示领域异常类型与面向人的消息。
        print("  单工具预算结果:", type(error).__name__, str(error))
    else:
        raise AssertionError("更短的工具预算应先耗尽")
    assert tool_resource.state is ResourceState.RELEASED

    # 第二个请求只剩 0.01 秒，工具预算却有 1 秒。工具不创建更晚的内层范围，而是让
    # 外层 timeout_at(request_deadline) 统一终止整条当前调用链。
    request_events: list[str] = []
    request_resource = FakeIOResource("request-deadline", request_events)
    request_deadline: float = loop.time() + 0.01
    try:
        async with asyncio.timeout_at(request_deadline):
            await tool_with_local_budget(
                request_resource,
                asyncio.Event(),
                asyncio.Event(),
```

```
684  async def demonstrate_absolute_deadlines() -> None:
714      try:
715          async with asyncio.timeout_at(request_deadline):
716              await tool_with_local_budget(
717                  request_resource,
718                  asyncio.Event(),
719                  asyncio.Event(),
720                  request_deadline=request_deadline,
721                  tool_budget=1.0,
722              )
723      except TimeoutError:
724          # 这个 except 位于请求 timeout_at 范围外，所以把异常解释为“整体请求超时”。
725          record(request_events, "请求边界：整体截止时间耗尽")
726      else:
727          raise AssertionError("整个 Agent 请求应在共同截止时间停止等待")
728
729      # 事件成员断言确认走的是“沿用请求”分支，而不是碰巧由另一个局部超时结束。
730      assert "预算：工具沿用请求的绝对截止时间" in request_events
731      assert request_resource.state is ResourceState.RELEASED
732
733
734  async def demonstrate_timeout_is_not_hard_interrupt() -> None:
735      # 这一反例说明 timeout 依赖事件循环调度取消回调，不能抢占正在运行的同步代码。
736      print_section("8. 超时不是硬中断：阻塞事件循环会让超时延后")
737      events: list[str] = []
738      loop: AbstractEventLoop = asyncio.get_running_loop()
739      # started_at 只用于生成近似观察值；最终断言只验证事件顺序，不验证精确耗时。
740      started_at: float = loop.time()
741
742      try:
743          async with asyncio.timeout(0.01):
744              record(events, "阻塞反例：进入 timeout 范围")
745              # 教学反例：当前线程阻塞时，事件循环无法执行到期回调。0.03 秒大于 0.01 秒，
746              # 但 time.sleep() 不会像 await 一样把控制权交还事件循环。
747              time.sleep(0.03)  # noqa: ASYNC251
748              record(events, "阻塞反例：同步阻塞先完成")
749              # 到这个 await 才重新出现协作调度机会；事件循环随后处理已经到期的取消。
```

```python
async def demonstrate_timeout_is_not_hard_interrupt() -> None:
    try:
        async with asyncio.timeout(0.01):
            record(events, "阻塞反例: 进入 timeout 范围")
            # 教学反例: 当前线程阻塞时，事件循环无法执行到期回调。0.03 秒大于 0.01 秒，
            # 但 time.sleep() 不会像 await 一样把控制权交还事件循环。
            time.sleep(0.03)  # noqa: ASYNC251
            record(events, "阻塞反例: 同步阻塞先完成")
            # 到这个 await 才重新出现协作调度机会；事件循环随后处理已经到期的取消。
            await asyncio.Event().wait()
    except TimeoutError:
        # 离开 timeout 范围后才得到 TimeoutError。elapsed 用 .3f 格式化为三位小数，
        # 只是帮助肉眼观察“晚于预算”，不同机器上的具体数值可以变化。
        elapsed: float = loop.time() - started_at
        record(events, f"阻塞反例: 之后才观察到 TimeoutError (约 {elapsed:.3f}s)")
    else:
        raise AssertionError("恢复协作等待后应观察到已经到期的超时")

    # 不断言 elapsed 的具体范围，只验证同步阻塞结束事件一定早于超时被观察到。
    blocking_finished: int = events.index("阻塞反例: 同步阻塞先完成")
    timeout_observed: int = next(
        index for index, event in enumerate(events) if "之后才观察到 TimeoutError" in event
    )
    assert blocking_finished < timeout_observed


async def shielded_owner(
    work_started: asyncio.Event,
    work_may_finish: asyncio.Event,
    caller_cancelled: asyncio.Event,
    task_holder: list[asyncio.Task[str]],
    events: list[str],
) -> None:
    """选学: 所有者保存被 shield 的 Task，并在自己取消后仍负责收集它。"""

    # 嵌套 async def 只在 shielded_owner 被调用时定义，且可直接读取外层参数 events 和 Event。
```

```
767  async def shielded_owner(
774      """选学：所有者保存被 shield 的 Task，并在自己取消后仍负责收集它。"""
775
776      # 嵌套 async def 只在 shielded_owner 被调用时定义，且可直接读取外层参数 events 和 Event。
777      # child 等待 work_may_finish，因此 owner 被取消时它仍处于未完成状态。
778      async def protected_work() -> str:
779          record(events, "shield child: 开始")
780          work_started.set()
781          await work_may_finish.wait()
782          record(events, "shield child: 正常完成")
783          return "受保护结果"
784
785      # owner 创建 child，所以 owner 负责保存和回收它。除局部变量外，再追加到调用方传入的
786      # task_holder 列表，使实验外层也持有强引用并能观察 child 的状态。
787      child_task: Task[str] = asyncio.create_task(protected_work(), name="shielded-child")
788      task_holder.append(child_task)
789      try:
790          # shield(child_task) 返回一个受保护的等待对象：owner 的取消会中断这个 await，
791          # 但不会沿这条等待边调用 child_task.cancel()。
792          await asyncio.shield(child_task)
793      except asyncio.CancelledError:
794          # owner 自己仍然收到取消。先通知实验外层，再继续履行对 child 的回收责任。
795          record(events, "shield caller: 自己仍收到 CancelledError")
796          caller_cancelled.set()
797          # shield 只隔离这一次等待的取消传播。调用方创建了 child，因此仍要
798          # 保存引用、决定结束策略，并真正 await 它以收集结果或异常。
799          # 直接 await 原 child，等它正常完成并收集结果；随后裸 raise 让 owner 保持取消终态。
800          result: str = await child_task
801          record(events, f"shield owner: 最终收集 {result}")
802          raise
803
804
805  async def demonstrate_shield_ownership() -> None:
806      # 本段同时观察两个独立终态：被 shield 的 child 正常完成，owner 仍以取消结束。
807      print_section("9. 选学：shield 保护被等待对象，不取消调用方")
808      events: list[str] = []
```

```python

async def demonstrate_shield_ownership() -> None:
    # 本段同时观察两个独立终态：被 shield 的 child 正常完成，owner 仍以取消结束。
    print_section("9. 选学：shield 保护被等待对象，不取消调用方")
    events: list[str] = []
    work_started = asyncio.Event()
    work_may_finish = asyncio.Event()
    caller_cancelled = asyncio.Event()
    # 可变列表充当最小“引用容器”；owner 会把自己创建的 child 放入其中。
    task_holder: list[asyncio.Task[str]] = []
    owner_task: Task[None] = asyncio.create_task(
        shielded_owner(
            work_started,
            work_may_finish,
            caller_cancelled,
            task_holder,
            events,
        ),
        name="shield-owner",
    )
    # 确认 child 已经启动并等待，再取消 owner。caller_cancelled 则确认 owner 已在
    # shield await 处收到 CancelledError，随后才读取 child 状态。
    await work_started.wait()
    owner_task.cancel("取消调用方")
    await caller_cancelled.wait()

    # 本例只创建一个 child，所以索引 0 取得那一个强引用。此时 shield 已阻断向内取消，
    # child 应仍是未完成且未取消。
    child_task: Task[str] = task_holder[0]
    print("  调用方已收到取消时，child.cancelled():", child_task.cancelled())
    assert not child_task.done()
    assert not child_task.cancelled()

    # 放行 child 后，owner 内部会先 await 并记录 child 的结果，再重新抛出自己的取消。
    work_may_finish.set()
    try:
        await owner_task
```

```
async def demonstrate_shield_ownership() -> None:
    # 放行 child 后，owner 内部会先 await 并记录 child 的结果，再重新抛出自己的取消。
    work_may_finish.set()
    try:
        await owner_task
    except asyncio.CancelledError:
        # 这是 owner 的外层所有者收集取消的边界。
        record(events, "shield 外层：调用方最终保持取消状态")
    else:
        raise AssertionError("shield 不应吞掉调用方自己的取消")

    # result() 只在已完成且未取消、未失败的 Task 上安全调用；前一条断言先验证这个前提。
    assert child_task.done() and not child_task.cancelled()
    assert child_task.result() == "受保护结果"
    assert owner_task.cancelled()


async def run_examples() -> None:
    # 顺序 await 让九段实验逐个完成；每段都自行收集预期异常和创建的 Task，因此不会把
    # pending 工作泄漏到下一段。任一断言失败都会立即中止，避免打印误导性的完成消息。
    await demonstrate_normal_completion()
    await demonstrate_cancellation_phases()
    await demonstrate_direct_await_propagation()
    await demonstrate_exception_hierarchy_and_swallowing()
    await demonstrate_timeout_context()
    await demonstrate_wait_for_cleanup()
    await demonstrate_absolute_deadlines()
    await demonstrate_timeout_is_not_hard_interrupt()
    await demonstrate_shield_ownership()
    print("\n所有第 21 章示例与运行时断言均已完成。")


def main() -> None:
    # asyncio.run() 创建事件循环、把 run_examples coroutine 驱动到完成，最后关闭事件循环。
    # 同一个进程中通常只应在同步入口调用它，不能在已运行的事件循环里再次嵌套调用。
    asyncio.run(run_examples())
```

```
805 async def demonstrate_shield_ownership() -> None:
850     assert owner_task.cancelled()
851
852
853 async def run_examples() -> None:
854     # 顺序 await 让九段实验逐个完成；每段都自行收集预期异常和创建的 Task，因此不会把
855     # pending 工作泄漏到下一段。任一断言失败都会立即中止，避免打印误导性的完成消息。
856     await demonstrate_normal_completion()
857     await demonstrate_cancellation_phases()
858     await demonstrate_direct_await_propagation()
859     await demonstrate_exception_hierarchy_and_swallowing()
860     await demonstrate_timeout_context()
861     await demonstrate_wait_for_cleanup()
862     await demonstrate_absolute_deadlines()
863     await demonstrate_timeout_is_not_hard_interrupt()
864     await demonstrate_shield_ownership()
865     print("\n所有第 21 章示例与运行时断言均已完成。")
866
867
868 def main() -> None:
869     # asyncio.run() 创建事件循环、把 run_examples coroutine 驱动到完成，最后关闭事件循环。
870     # 同一个进程中通常只应在同步入口调用它，不能在已运行的事件循环里再次嵌套调用。
871     asyncio.run(run_examples())
872
873
874 # 导入本模块时 __name__ 是模块名，只定义函数和类；直接执行文件时它才等于 "__main__"，
875 # 因而调用同步入口 main()。这让示例既可独立运行，也可被其他代码导入检查而不自动执行。
876 if __name__ == "__main__":
877     main()
878
```